



PARACODING

Fleet Control Plane

Security Review

Fleet control plane, privileged executor,
credential surfaces and the release gate

Date	19 September 2026
Tree	55d12120c52eb4ac932d72fb7cfea5851065d9b3
Findings	30 (2 critical · 9 high · 15 medium · 4 low)
Method	Source review against the bytes at that commit
Status	For operator triage — no finding fixed yet



What this review found

Thirty findings, every one cited to a file, a line number and the literal source line at **55d12120**. Nothing here is inferred from a comment: this codebase documents its own past incidents at length, and a comment asserting that something is safe was treated as a claim to check rather than evidence. Where a defect could not be confirmed from the bytes it is marked as such in its own entry.

CRITICAL	HIGH	MEDIUM	LOW
2	9	15	4

The approval chain does not authorise anything

On the shipped default the control plane signs the command the caller just handed it, and the executor then verifies that signature against a key the signer holds. Every rung below — the canon, the key allowlist, the single-use claim, the create-only journal — is correctly built and rests on nothing. The executor compounds it by running jobs with its own credentials, which hold project-wide `roles/run.admin`, so a job can remove the controls that gated it. **SEC-01, SEC-02.**

Several brakes exist but cannot be applied

The lockout checker is nine correct rules the installer never arms on the executor. The per-job passkey check refuses every job when switched on, because the challenge it needs is never sent. The acknowledgement that overrides a lockout is a flag the calling agent sets on itself. Each reads as a control in the documentation and is not one. **SEC-03, SEC-04, SEC-18, SEC-19, SEC-20.**

Ownership is enforced on the roster, not on the surfaces that act

The strain ownership model added this month is checked when drawing the sidebar and not when a route runs a tool, mints a key, or rewrites a strain document. Three routes reach strains their caller does not own. This matters more now than last week, because multi-user is the reason the model exists. **SEC-07, SEC-08, SEC-12.**

One clean external-attacker path

Dynamic client registration is public and stores redirect URIs without validating them, and every later check compares against that same attacker-written list. It ends in an authorization code delivered to a host the attacker chose. It is the only finding here that needs no prior access at all. **SEC-05.**

The gate can promote on evidence it did not gather

A disabled IAP that never came back cannot fail a build, because the field recording it is written and never read. Production promotes an image no smoke run judged. A selftest row that proves nothing still counts as proof. The gate is unusually well built — these are the seams where it stops measuring and starts assuming. **SEC-10, SEC-11, SEC-23, SEC-25.**

Scope and method

Four independent passes over the tree at **55d12120**: the control plane's identity, admission and tool-class enforcement; the privileged executor and the approval chain; the OAuth server, credential storage and the lake cryptography; and the release gate, the judge and the installer generator. Roughly 25,000 lines were read



directly, concentrated on the paths that decide who may do what. Every candidate finding was then re-verified against the source before filing — several were discarded at that step, and two survived only in narrowed form. Generated output under `oss/release/` was read only to compare against its generator and is not cited as a defect site.

Not covered: runtime behaviour, live IAM beyond what the code and installer declare, dependency supply chain, and anything requiring execution against production. This is a source review.



Findings

Ranked by severity. Each carries the work item it was filed as.

CRITICAL **SEC-01**

PC_AUTO_APPROVE makes the whole approval chain self-referential

control-plane/src/index.ts:1700 · oss/gen.py:2046

```
cmd_sha: cmdSha, approved_sha256: cmdSha, approved_sha256_at: new Date().toISOString(),
```

CLAIM	With auto-approve on, the control plane computes the command pin and the KMS signature from the command the caller just supplied, in the same request that fires the job.
WHY IT MATTERS	The executor verifies that pin against itself and that signature against a key the signer holds, so the chain proves only that the control plane restated the caller's bytes. Any agent holding the stage tool class reaches arbitrary shell with a valid, signed, journalled approval naming auto:lockout-check as the approver. The only content-based refusal on the path is a five-regex check for Secret Manager deletes.
DIRECTION	A decision, not a patch. Either accept this as the posture for a single-operator fleet and say so in SECURITY.md in these words, correcting every doc that claims approvals are verified; or default PC_AUTO_APPROVE to 0 with a real approver. Changing the default without warning stops every agent in the fleet.
CONFIDENCE	Confirmed. PC_AUTO_APPROVE="\${PC_AUTO_APPROVE:-1}" is the shipped installer default, so this is the normal state of every install.

Work item ahmK20Z05hWRMRcJs5qd · assigned **fleet-security**

**CRITICAL** SEC-02**Jobs run with the executor's own token, which holds project-wide roles/run.admin**

gate-exec/exec_server.py:1717 · oss/gen.py:3096

```
env["CLOUDSDK_AUTH_ACCESS_TOKEN"] = _pc_metadata_token()
```

CLAIM	When no approver token is forwarded — every job on the auto-approve path — the executor hands the child process its own service-account credentials.
WHY IT MATTERS	That service account is granted roles/run.admin at PROJECT scope by the installer. A job body can therefore rewrite the Cloud Run configuration of the executor and the control plane themselves: one `gcloud run services update` removes the signature verifier, the key allowlist and the PATH jail on the next revision. The same token carries secretAccessor on the passkey credential store.
DIRECTION	Scope the grant to the services the executor legitimately deploys, or drop to run.developer with explicit per-service bindings. Separately, reconsider whether a job needs the executor's identity at all when no approver token was forwarded.
CONFIDENCE	Confirmed. The installer's own comment concedes the grant lets the executor modify or delete the control plane's services and that nothing but the prompt enforces otherwise.

Work item ySN0lhrPo6NQsq6zbTf5 · assigned **fleet-security****HIGH** SEC-03**PC_GUARDRAILS is never written to gate-exec, so the lockout checker can never be armed**

oss/gen.py:4573, 4593 · gate-exec/exec_server.py:1600

```
_lc_guard = os.environ.get("PC_GUARDRAILS", "0").strip() == "1"
```

CLAIM	The installer propagates the guardrail flag to the console and the MCP surface and not to the executor — the one service that runs privileged commands.
WHY IT MATTERS	An operator who deliberately sets PC_GUARDRAILS=1 gets brakes on two of three services. Every lockout-class body, including the run services update that would strip the executor's own controls, is journalled as exec_lockout_class_ran and then executed. Nine correct rules that nothing on this install can turn on.
DIRECTION	Add PC_GUARDRAILS=\$PC_GUARDRAILS to the gate-exec --set-env-vars in oss/gen.py, regenerate, and set it on the live service. oss/release/ is generated — fix the generator, never the copy.
CONFIDENCE	Confirmed by exhaustive grep of oss/gen.py: the flag appears in the console deploy, the mcp deploy and prose, and in no gate-exec deploy line.

Work item ZYAIY5tCXCTfEgPjtJ0L · assigned **fleet-infra**



HIGH

SEC-04

The operator acknowledgement that overrides a lockout refusal is set by the calling agent

control-plane/src/index.ts:3003

```
const _jargs: any = _confirm ? { command: a.command, lockout_ack: true } : { command: a.command };
```

CLAIM	run_command takes a confirm flag from the agent; the control plane writes lockout_ack:true into the job arguments before signing, so it lands inside the signed hash and verifies cleanly.
WHY IT MATTERS	The executor then journals exec_lockout_acked — "carried a signed operator acknowledgement, so it RAN" — for a body no human ever saw. The executor's comment argues only that lockout_ack cannot be added in transit, which is true and beside the point: nothing binds it to a person.
DIRECTION	lockout_ack must be derivable only from a path a human actually touched — a console session, a passkey assertion, a pending_confirm click — never from a tool argument.
CONFIDENCE	Confirmed.

Work item 1mnJHq2wiHRfIMT9NRy5 · assigned **fleet-security**

HIGH

SEC-05

POST /oauth/register is public and stores redirect_uris with no validation

control-plane/src/index.ts:13272

```
const redirect_uris = Array.isArray(body.redirect_uris) ? body.redirect_uris.map((x: any) => String(x)).filter(Boolean) : [];
```

CLAIM	Dynamic client registration is unauthenticated by design and accepts redirect URIs as arbitrary strings; every later check compares against that same attacker-written list, so it always passes.
WHY IT MATTERS	An attacker registers a client pointing at their own host, sends an approver an ordinary-looking /oauth/authorize link, and receives the authorization code when the operator consents — then exchanges it for a live fleet bearer. A javascript: URI instead ends in script execution on the console origin, since the consent page assigns the redirect to window.location.
DIRECTION	Validate at registration: require https (or http://localhost for development), reject any other scheme, reject embedded credentials and fragments, and consider an operator allowlist of registrable hosts.
CONFIDENCE	Confirmed. This is the highest-value finding for an attacker with no prior access.

Work item U0KKA8qY4qDUTULtDs8o · assigned **fleet-security**



HIGH

SEC-06

The Gemini Enterprise seat OAuth flow sends no state parameter

control-plane/src/index.ts:9583

```
client_id: GE_CLIENT, redirect_uri: geRedirect(req), response_type: 'code',
```

CLAIM	/ge/connect builds the Google authorization URL with no state, and /ge/callback never reads one — an exhaustive grep for req.query.state across the repository returns zero matches.
WHY IT MATTERS	An attacker completes the flow in their own browser, captures their own code, and gets a console-authenticated operator to load that callback URL. The handler exchanges the attacker's code and writes the attacker's refresh token into ge_seat under the victim's seat key; the victim's Gemini Enterprise traffic then runs on, and is visible to, the attacker's account.
DIRECTION	Mint a random state, bind it to the seat key and session, store it server-side with a short TTL, refuse a callback whose state is missing or wrong. The MCP authorize path already does this at index.ts:13452.
CONFIDENCE	Confirmed. Needs an authenticated victim to open a link — ordinary CSRF preconditions, not a mitigation.

Work item Be8NKXLpWgG5fQBJs9LU · assigned **fleet-security**

HIGH

SEC-07

POST /api/chat builds the full admitted toolset for any agentId the body names

control-plane/src/index.ts:11379, 11458

```
const toolset: HarChatTool[] = agentId ? await harChatToolset(agentId) : [];
```

CLAIM	agentId is read straight from the request body and checked only against a charset regex; the handler then loads that strain's history and builds its full toolset.
WHY IT MATTERS	run_command, gcp_api, git_push and stage_privileged_job all execute closed over that strain's identity, journalled and staged_by as that strain. pcStrainVisible — the owner/shared_with rule — is consulted when building the sidebar roster and never here, so any console user can act as, and read the private conversation of, a strain they do not own.
DIRECTION	Resolve the viewer with pcStrainViewer and refuse unless pcStrainVisible passes, before either the history read or the toolset build.
CONFIDENCE	Confirmed. This is the finding that makes the ownership model cosmetic: sharing is enforced on the roster, not on the surface that does things.

Work item 574BGfNro9Z6iaeHmuKQ · assigned **fleet-security**



HIGH

SEC-08

POST /api/strains/provision can rewrite any strain, making owner-only sharing unenforceable

control-plane/src/index.ts:12748, 12867

```
if (!waSessionOk(req)) { res.status(401).json({ error: 'unlock first' }); return; }
```

CLAIM	The route is gated on a console session and nothing else, yet merge-writes owner_email, oauth_client_id, oauth_email, sa_email, user_email, delegating, tool_classes and status onto any strain the caller names.
WHY IT MATTERS	A console user can stamp themselves as owner of someone else's strain, install a connector binding so their keyless chats resolve to it, widen a deliberately fail-closed strain's tool classes, set user_email to take over the delegated Gemini lane, or flip a retired strain back to active. /api/strains/retire has the same gate and no role regex at all.
DIRECTION	Refuse to modify an EXISTING strain the caller does not own, using the same rule /api/strains/share applies; stamp owner_email from the verified viewer on create only.
CONFIDENCE	Confirmed. This is why the new owner-only share route cannot hold on its own.

Work item 9dXmxShvrYEH1g5dB6z · assigned **fleet-security**

HIGH

SEC-09

Production leaves a permanent tag URL on an MCP revision the gate refused

pipeline/cloudbuild-prod.yaml:451

```
--no-traffic --tag=p${_SHORT} \
```

CLAIM	The MCP revision gets a durable public tag URL at deploy time, before any verdict exists, and no pipeline step ever removes it.
WHY IT MATTERS	When the readback step refuses that revision it prints that both surfaces are untouched — but the MCP service carries allUsers invoker, so the rejected revision stays anonymously reachable at its tag URL. The specific hazard is a revision registering the full console route table on the public, IAP-free surface.
DIRECTION	Remove the tag on the refusal path, or stop tagging no-traffic revisions on the public surface and address them by revision name.
CONFIDENCE	Confirmed. --remove-tags appears only in deploy runbooks, never in any pipeline file.

Work item t0dzju7QwiWS3TLHDxow · assigned **fleet-infra**



HIGH

SEC-10

The collector turns IAP off, records whether it came back, and nothing reads that field

pipeline/collect-evidence.py:2817

```
ev["iap_restore_confirmed"] = bool(
```

CLAIM	IAP is disabled for the unauthenticated probes and re-enabled in a finally block whose return value is discarded; the confirmation is written into the bundle and never consulted.
WHY IT MATTERS	If the restore fails, or the step is killed before the finally runs, the console is left with no IAP in front of it, the collector still exits 0, smoke still reports VERDICT 0, and the gate promotes. <code>iap_enabled_after_restore</code> , <code>iap_disable_never_propagated</code> , <code>iap_toggle_refused</code> and <code>config_changes</code> are equally unread.
DIRECTION	Assert on it in <code>smoke.py</code> and add its literal to <code>promote-gate.sh</code> 's <code>COUPLED_LITERALS</code> so the check cannot silently stop matching.
CONFIDENCE	Confirmed. Exhaustive grep: two occurrences, the write and its generated copy.

Work item n0T6Jiqgo0cUecZt7VMz · assigned **fleet-infra**

HIGH

SEC-11

Production promotes an image no smoke run ever judged

pipeline/cloudbuild-prod.yaml:402, 407

```
WANT=$(python3 -c "import  
json;print(json.load(open('/workspace/devbuilt.json')).get('image_digest',''))")
```

CLAIM	The digest-match step reads a key the dev marker never writes, so the comparison is empty on every promotion and the warning branch always runs; the mismatch exit is commented out.
WHY IT MATTERS	Production then builds its own image with unpinned npm install and promotes that, while the only evidence of safety is a verdict computed against a different image built by a different command into a different repository. A floating dependency ships unjudged and the log prints a reassuring line.
DIRECTION	Emit <code>image_digest</code> in the dev marker and make a mismatch a hard refusal, or stop rebuilding in prod and promote the artifact that was judged.
CONFIDENCE	Confirmed. The file states this as the current condition, not as a fixed defect.

Work item sLgCNJaAyKx5gKmeXJQR · assigned **fleet-infra**



MEDIUM

SEC-12

/api/sessions/mint issues a live seven-day key for any pasteable strain

control-plane/src/index.ts:15070

```
if (srow.pasteable !== true) { res.status(403).json({ error: 'role is not pasteable: ' + role });  
return; }
```

CLAIM	The last check before a credential is generated is a per-strain flag, not a per-caller ownership check.
WHY IT MATTERS	A console user mints a key for someone else's strain and pastes it anywhere; every tool call resolves to and is attributed to that strain for seven days. /api/sessions/roleflags is also console-only, so a caller can first set pasteable on a strain the operator left unpasteable.
DIRECTION	Apply the owner/shared_with rule before minting, and before flipping role flags.
CONFIDENCE	Confirmed.

Work item fUufujiLGMMZpoNiQLvV · assigned **fleet-security**

MEDIUM

SEC-13

The in-process user-token cache skips the audience and agent pins the durable session enforces

control-plane/src/index.ts:14701

```
let email: string | null = (cc && (now - cc.at) < 300000) ? cc.email : null;
```

CLAIM	The five-minute memory cache is keyed on the token hash alone; the Firestore branch one line below checks that the record names the same agent strain and the same audience, and the memory branch checks neither.
WHY IT MATTERS	With two delegating strains on different OAuth clients, a token verified once through strain A is honoured for five minutes when replayed through strain B, whose audience it was never minted for.
DIRECTION	Key the cache on (token hash, agentRole, aud), or store both in the entry and compare them on a hit.
CONFIDENCE	Confirmed in the code path. Whether this install currently runs more than one delegating strain was not checked; the defect is in the path regardless.

Work item 9aNPnm3LLmpPcdp272Jq · assigned **fleet-security**



MEDIUM

SEC-14

/git/blob and /git/release-tree apply no tool-class check, so a read-only key can move a branch

control-plane/src/index.ts:14196

```
const who = await pcUploadCaller(req);
```

CLAIM	Both routes resolve the session key and stop there; GET /git/archive was deliberately changed to also require the read class, and these two write routes make no equivalent call.
WHY IT MATTERS	A key narrowed to ['read'] cannot register git_propose or git_push, both class write. The holder posts the bytes to /git/blob and then posts /git/release-tree to propose a commit and move the release branch — the narrowing is escaped by using the HTTP route instead of the tool.
DIRECTION	Resolve effective classes in pcUploadCaller and require write on both routes.
CONFIDENCE	Confirmed. Bounded by PC_RELEASE_TREE_PREFIX being configured.

Work item Xpk9vosYEaFEUq4faTK3 · assigned **fleet-security**

MEDIUM

SEC-15

On a bound connector, omitting the agent argument escapes a key's tool-class narrowing

control-plane/src/index.ts:14776

```
if (bid.oidc) return await pcDelegatedUser(req, bearer, x.id);
```

CLAIM	Dropping the agent argument skips the key branch entirely; because the operator bound this connector to a strain, the keyless branch admits the call as that strain with no narrowing.
WHY IT MATTERS	A subagent given a key narrowed to ['read'] simply stops presenting it and gets the strain's full class set, including stage. The narrowing holds only for a caller that chooses to keep presenting its key, which is not a security property.
DIRECTION	A keyless tools/call on a bound connector should resolve to the narrowest class set, or be refused, when PC_SESSION_ENFORCE is on.
CONFIDENCE	Confirmed.

Work item v4nEKbCeT2RnLdeseqd7 · assigned **fleet-security**



MEDIUM

SEC-16

A Gemini Enterprise user token becomes a seven-day session that outlives Google revocation

control-plane/src/index.ts:14716

```
db.collection('user_sessions').doc(th).set({ email, aud, agent: agentRole, verified_at: now, exp: now + PC_USER_SESSION_TTL_MS, ...
```

CLAIM	The first time a raw Google access token passes tokeninfo, a seven-day server-side session is minted from it and later calls skip the Google check entirely.
WHY IT MATTERS	Anyone who obtains those token bytes once — a proxy log, an agent request dump, a shared screen — can keep presenting a token Google has already expired or revoked and be admitted as that person for the rest of the TTL.
DIRECTION	A decision, not a defect: the comment states the trade deliberately because GE does not reliably refresh. Options are a materially shorter TTL, periodic re-verification inside the TTL, or binding the session to something the holder cannot replay alone.
CONFIDENCE	Confirmed.

Work item Ku0ndE7asU9PMvdmUaj · assigned **fleet-security**

MEDIUM

SEC-17

The refresh grant does not rotate the refresh token

control-plane/src/index.ts:13653

```
await oaSet('oauth_refresh', rtInH, { role: rr.role, email: rr.email, client_id: rr.client_id, issued_at: ..., exp: rexp, access_hash: atH, revoked: false });
```

CLAIM	The access token rotates; the refresh token is rewritten at the same document id and no new one is returned.
WHY IT MATTERS	Clients are registered with no authentication method, so the refresh token is the whole credential. An attacker who captures one mints fresh access tokens for its full thirty-day life, the legitimate client keeps working, and the server cannot distinguish the two holders or detect the theft.
DIRECTION	Issue a new refresh token on every refresh, revoke the old one, and treat reuse of a rotated token as a compromise that revokes the chain — the OAuth 2.1 guidance for public clients.
CONFIDENCE	Confirmed.

Work item cVbFLDYjto95lsFSdaeH · assigned **fleet-security**



MEDIUM

SEC-18

PC_REQUIRE_ASSERTION=1 refuses every job, because no challenge is ever sent

control-plane/src/index.ts:5634 · gate-exec/exec_server.py:1254

```
if not _assn or not _chal:
```

CLAIM	The executor reads expected_challenge from the request body and the control plane never sends that field — it occurs only under gate-exec/ and in two wiki pages.
WHY IT MATTERS	Arming the flag returns 428 for every job and bricks the install, so the operator rolls it back. The one control genuinely independent of a compromised control plane cannot be turned on.
DIRECTION	Send a real challenge when staging, or delete the half-built path so nobody believes the control exists.
CONFIDENCE	Confirmed. Known-but-unfixed: a wiki page records the same fact.

Work item X25NtM0UiP1z3Zinc036 · assigned **fleet-security**

MEDIUM

SEC-19

The passkey verifier checks no origin and no signature counter, so an assertion is replayable

gate-exec/pcwebauthn.py:46

```
flags = ad[32]
```

CLAIM	Parsing stops at the flags byte; the signature counter at bytes 33–37 is never read, and clientData.origin is never checked.
WHY IT MATTERS	The challenge compared against is supplied by the caller with no server-side record that it was issued; the binding check only asserts the trailing bytes equal a hash the caller can construct. One captured assertion for a given job and action is reusable indefinitely.
DIRECTION	Fix alongside SEC-18: check origin against an allowlist, read and persist the counter per credential, issue challenges server-side with a single-use record.
CONFIDENCE	Confirmed. Currently unreachable because of SEC-18, which is the only thing bounding it.

Work item gU5aauyFvhVC8UJASdyp · assigned **fleet-security**



MEDIUM

SEC-20

The executor PATH jail links interpreters, so it constrains nothing

gate-exec/exec_server.py:1134, 1697

```
env["PATH"] = _jail_dir
```

CLAIM	The allowlist of symlinked binaries includes python3, python, bash, sh, env, xargs and find; any one defeats a PATH restriction outright.
WHY IT MATTERS	python3 -c with subprocess, or env with a rewritten PATH, runs exactly the binaries the jail was built to block. The file concedes an absolute path still runs; it does not concede that a listed interpreter makes the control decorative, and the tool description shipped to every agent calls it a real control.
DIRECTION	Correct the tool description and the docs to state what the jail actually does, then decide whether a real boundary is wanted. The per-request jail directory is also never cleaned up.
CONFIDENCE	Confirmed.

Work item FFo3FkIfikvmSLwDZaEu · assigned **fleet-security**

MEDIUM

SEC-21

The token a job runs as is bound to nothing in the approval

gate-exec/exec_server.py:1672

```
access_token = body.get("access_token")
```

CLAIM	The token is read from the request body, covered by neither approval canon, and never compared against the approver named in the verified signature.
WHY IT MATTERS	A caller who can reach /run pairs a genuinely signed approval naming a human with any token they hold, and the journal records the run as that human's. Note also that the command pin covers only the command string — not argv, not the environment, not the working directory, not the interpreter.
DIRECTION	Cover the token's identity in the canon, or verify its email against the signed approver before running.
CONFIDENCE	Confirmed. Reaching /run needs invoker on gate-exec, granted only to the control-plane SA, so this is defence-in-depth against the adversary the design names.

Work item M0zOyheo3vzkkfLbFYmz · assigned **fleet-security**



MEDIUM

SEC-22

The promotion re-read never checks that the serving revision is the one that was judged

pipeline/verify-serving.py:71

```
serving_revs = [t for t in traffic if t.get('percent', 0) > 0]
```

CLAIM	The confirmation resolves whatever revision holds traffic and passes on an image digest and a commit var; the target revision's name is never passed in or compared.
WHY IT MATTERS	A concurrent traffic move — a rollback, a console click, a second build of the same commit — landing between the move and the re-read leaves a different revision serving while the gate prints PROMOTED and CONFIRMED BY RE-READ.
DIRECTION	Pass the target revision name into verify-serving.py and require the serving revision to equal it.
CONFIDENCE	Confirmed. This discards at the confirmation step what the file establishes everywhere else.

Work item TxqfGzMg3citacxLJ6Uf · assigned **fleet-infra**

MEDIUM

SEC-23

A vacuous selftest row still counts as proof that a check can fail

devgate/smoke.py:2443

```
bites = (got == expect) if expect is not None else (got != "FAIL")
```

CLAIM	When the finding already held the expected status before seeding, the row is flagged vacuous and still returns bites, so it is counted in the N/N proved-they-can-fail census.
WHY IT MATTERS	The verdict chain branches on the dead-check count and never on the vacuous count, and the promote gate has no grep for VACUOUS. A control that proves nothing is scored as proof, the ratio stays N/N, and the run can still reach VERDICT 0 and promote — the exact shape the selftest exists to catch.
DIRECTION	Make the vacuous count a first-class verdict input, print it in a line the gate scrapes, and add that literal to COUPLED_LITERALS.
CONFIDENCE	Confirmed in the code. Whether any control today hits this shape was not enumerated.

Work item hYlJJ2uTxS9FtI1MRHlm · assigned **fleet-infra**



MEDIUM

SEC-24

The installer writes the session secret in cleartext and does not verify its removal

oss/gen.py:3366, 3376

```
openssl rand -base64 32 | tr -d '\n' > "$HERE/.s.tmp" || PC_MK_RC=$?
```

CLAIM	The console session secret is written into the unpacked release directory under the ambient umask with no chmod, and removed by an unchecked python3 call.
WHY IT MATTERS	If python3 is absent, or the path contains an apostrophe, the removal fails silently, the script prints created and continues, and the key that mints console sessions sits world-readable on disk indefinitely. The same generator does chmod 0600 elsewhere, so the pattern is known and simply missing here.
DIRECTION	umask 077 before the write or chmod 600 after, use rm -f, and die if the file still exists.
CONFIDENCE	Confirmed.

Work item oLa3vzyZde4CCQUJruyf · assigned fleet-infra

MEDIUM

SEC-25

The judge's only route to verdict 0 requires an installer boundary the pipeline hardcodes

pipeline/cloudbuild-dev.yaml:740

```
SMOKE_INSTALL_EXIT=20 \
```

CLAIM	smoke.py reaches its zero verdict only when the install exit is 20, and the pipeline passes 20 as a literal while running no installer at all.
WHY IT MATTERS	The report prints that 0 requires BOTH the installer reaching the 9/10 boundary AND every functional assertion passing; the first half is true by assignment. The promote gate accepts the resulting VERDICT 0 and production promotes on that word.
DIRECTION	Run the installer in the lane and pass its real exit code, or drop the install half of the conjunction and stop printing a claim the run does not test.
CONFIDENCE	Confirmed.

Work item g4L4yELwPMdYqQLguLjr · assigned fleet-infra



MEDIUM

SEC-26

The IAP-toggle project fence is handed the project it is meant to fence

pipeline/cloudbuild-dev.yaml:672 · pipeline/collect-evidence.py:2514

```
--iap-toggle-projects "$$PROJECT" --firestore-db "$$FIRESTORE_DB"
```

CLAIM	The collector refuses to disable IAP when the lane's project is not in the allowlist, and the pipeline passes that same project as the allowlist, so the fence can never fire.
WHY IT MATTERS	Under the single-project layout, dev and prod are lanes of one project — the case the fence's own comment says it can no longer answer. The service acted on is an overridable substitution, so a trigger naming the production console would take IAP off the live console for the probe window with no refusal anywhere.
DIRECTION	Fence on the SERVICE name rather than the project, and refuse outright when the named console service is the production one.
CONFIDENCE	Confirmed for the inert fence. The substitution escalation is configuration-reachable, not observed — no such trigger exists in the tree.

Work item przLHCyFVspP612oyntf · assigned **fleet-infra**

LOW

SEC-27

Re-authorization retires at most 400 prior credentials and swallows every failure

control-plane/src/index.ts:13563

```
.limit(OA_RETIRE_MAX).get();
```

CLAIM	The retirement sweep is capped and called fire-and-forget, so the outcome is never surfaced.
WHY IT MATTERS	An account past the cap re-authorizes expecting old credentials to die; page order decides which are retired, the rest stay live, and the sign-in reports success either way. This install carries 118 token records and grows one per consent.
DIRECTION	Page until exhausted, or report the residue to the operator rather than discarding it.
CONFIDENCE	Confirmed in the code; the live record count per account was not measured.

Work item RHdEcj8fwzh10Q0Uz514 · assigned **fleet-security**



LOW

SEC-28

The lake decryptor rebuilds the AAD with a hardcoded flags byte

control-plane/src/index.ts:8765

```
const aad = Buffer.concat([blob.slice(0, 4), Buffer.from([epoch]), Buffer.from([0x00]),  
Buffer.from(path, 'utf8')]);
```

CLAIM	The flags position in the additional authenticated data is a constant rather than the byte read from the envelope, so the stored flags byte is not bound by the GCM tag on this path.
WHY IT MATTERS	An actor with write access to the lake bucket can flip that byte and the tag still verifies. The sibling implementation in vault-objenc.ts binds it correctly and would reject the same object — two codecs that are supposed to agree, silently diverging.
DIRECTION	Read the stored flags byte into the AAD in both implementations.
CONFIDENCE	Confirmed. No consumer reads flags today, so there is no exploit — it becomes one the moment a flag is given meaning.

Work item YZUo9DqqXdyASq1N5Cad · assigned **fleet-security**

LOW

SEC-29

A revoked session key keeps working for up to 60 seconds on instances that did not serve the revocation

control-plane/src/index.ts:14559

```
if (hit && (now - hit.at) < PC_SESS_TTL_MS) return hit.v;
```

CLAIM	Lookups are served from a sixty-second positive in-process cache and the revoke handler clears only the local map.
WHY IT MATTERS	On a service running at twenty instances, revoking a leaked key leaves it authenticating on every other warm instance until that entry ages out. Bounded, but revocation is the one operation where eventually is the wrong semantics.
DIRECTION	Write a tombstone the lookup consults, shorten the positive TTL for keys, or document the window explicitly in the revoke response.
CONFIDENCE	Confirmed.

Work item ZJcDe2hEI9cB3R9zuumc · assigned **fleet-security**



LOW

SEC-30

The `gate_session` cookie is verified but never minted, so every console user is a full approver

`control-plane/src/index.ts:4691`

```
const c = waCookie(req, 'gate_session'); if (!c || c.indexOf('.') < 0) return false;
```

CLAIM	<code>waMakeSession</code> has exactly one occurrence — its own definition — and no route sets the cookie, so console admission is IAP plus the approver list and nothing else.
WHY IT MATTERS	Fail-closed, so nobody is wrongly admitted. It matters because it is why SEC-07, SEC-08 and SEC-12 escalate only between approvers, and because it makes the approver gate on <code>/api/oauth/pending</code> a check that can never fire.
DIRECTION	Either mint the cookie and make the invited-user tier real, or delete the dead branch and the checks that depend on a distinction the console does not draw.
CONFIDENCE	Confirmed for the dead branch. Whether an out-of-tree component still sets the cookie against the same secret was not verifiable from the repository.

Work item 5PIUm5GHQTLVUD6C4r1v · assigned **fleet-security**



Examined and found sound

Recorded because a review that lists only defects tells you nothing about coverage. Each of these was read and not faulted.

Signature verification itself

The canon id selects the rebuild and is never parsed from the signature; the key version is compared against an allowlist with a constant-time compare and an empty allowlist returns unverifiable rather than passing; job id, command hash and argument hash are taken from the job about to run, never from the stamp. Length-prefixed canonicalisation makes separator collisions unconstructible, and the JSON serialiser refuses non-integral floats, unsafe integers and surrogates rather than approximating them.

Single-use execution and the audit trail

The claim object uses create-if-absent generation matching, the three-way created / exists / unknown result is never collapsed, and every refusal sits above the claim so a failed substitution burns no approval. Journal writes are create-only, so the executor cannot edit its own record.

Identity resolution

The attested service-account path pins the audience, requires an active strain and fails closed when unconfigured. IAP assertions are verified against the live key set with issuer, expiry and audience checked, and the plain email header is never trusted. A presented but unrecognised session key is a hard denial that never falls back to a weaker role.

Tool withholding

A withheld tool gets a refusal handler in both the key-narrowing and the enforcement branch, and the registry records the wrapped handler — so no dispatcher, chat surface or forwarding tool can reach a real handler the class check refused. Class narrowing is subset-only for every input including malformed ones.

Credential hygiene

Every credential is generated from a cryptographic source; only hashes are ever document ids; the key is returned once and deliberately kept out of the journal. PKCE is required, the authorization code is consumed before validation, and the redirect is re-compared at exchange. No wildcard CORS with credentials.

Lake cryptography

AES-256-GCM with a per-object derived key, fresh nonce per seal, the path bound into the authenticated data and the epoch read from the object rather than the current write epoch. The 34-byte overhead the documentation claims is exactly what the construction produces.

Gate coupling

Every literal the promotion gate scrapes out of the judge is still present in the judge, including indentation — a rename would fail loudly rather than degrade into a silent no-op. The gate proves it can still refuse on every build by running itself over a red fixture and requiring one exact exit code, and over a refused-collection fixture requiring another.

Installer IAM

No owner or editor grants, no broad storage admin, no token-creator; signing rights to the control plane and public-key read to the executor; service-account-user confined to the two named runtime identities. The public



invoker grant is confined to the MCP surface. The single exception is the run.admin grant, filed as SEC-02.

A note on what is not a defect

Two of the most serious entries — SEC-01 and SEC-16 — describe behaviour the code documents as a deliberate trade. They are filed anyway, because a decision taken once in a comment is not the same as a decision an operator is still making on purpose. Both ask for a ruling rather than a patch, and neither should be changed without one: switching auto-approve off stops every agent in the fleet.